main

April 8, 2026

1 Homework 3

Some admin stuff:

- From a code perspective, we prefer modular development, so we had files for each part of the process. All code is pasted at the bottom in the Appendix.
- Runnable on a colab GPU. Just adjust `COLAB_DIR = Path("/content/drive/MyDrive/path/to/root/folder")` in `config.py` as per your requirements

[ ]:
```
import re
import json
from dataclasses import asdict
from IPython.display import Image

import pandas as pd
import torch

from config import (
    DataConfig,
    GlobalTrainingConfig,
    RunConfig,
    TokenConfig,
    TokenizationConfig,
    ModelConfig
)
from models import TokenChunkDataset, load_model
from tokenizer import (
    build_token_memmap,
    build_tokenizer,
    iter_stories,
    count_tokens,
    get_token_vector,
    find_closest_tokens,
)
from results import load_training_result, load_generations,␣
↪generations_to_dataframe
from training import train_model
```

```
from utils import save_json
from plot import plot_training_curves, plot_validation_curves

pd.set_option("display.max_colwidth", None)
pd.set_option("display.max_rows", None)  # optional
pd.set_option("display.max_columns", None)  # optional
pd.set_option("display.width", None)  # optional
pd.options.display.float_format = "{:,.2f}".format
```

The ultimate torch device variable

[18]:
```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(device)
if device.type == "cuda":
    print(torch.cuda.get_device_name(0))
```

```
cpu
```

The config objects make our life a lot easier – one central place to manage all the different configuration values

[84]:
```
token_config = TokenConfig()
data_config = DataConfig()
global_training_config = GlobalTrainingConfig()
tokenization_config = TokenizationConfig()
run_config = RunConfig("hw3-submission")

# load all the results and the most recent model
results = {
    name: load_training_result(run_config.metrics / f"{name}.json")
    for name in [
        "1_base_steps=1k_vocab=3k",
        "2_base_steps=100k",
        "3_large-plus",
        "4_large-plus_vocab=5k",
        "5_xlarge-plus",
        "6_large-plus_context=512",
    ]
}
print("Loaded results")

model = load_model(run_config.models / "5_xlarge-plus.pt", device)
print("Loaded model")
```

```
Directory already exists: /Users/sanch/Obsidian/2025-
spring/Notes/242B/Homeworks/hw3/artifacts/runs/hw3-submission
Loaded results
Loaded model
```

1.1 (a) Tokenizer

We trained a BPE tokenizer with vocabulary size 3,000 on the first 1,000,000 training stories. BPE starts from a small inventory of characters or character-like pieces and repeatedly merges frequent adjacent pairs, so common fragments become single tokens while rare words still fall back to smaller subwords. This is preferable to pure character tokenization because the sequence length becomes much shorter, and it is preferable to full word tokenization because rare words and misspellings can still be represented without an exploding vocabulary.

For this notebook we used a metaspace BPE setup so spaces are preserved on decode, and we reserved the special tokens `<pad>`, `<bos>`, `<eos>`, and `<unk>`.

We also printed out 5 sentences from the first story, looked at the word count vs. token count, to see what tokens the tokenizer was picking up. The outputs are below

[29]:
```
# build_tokenizer will load the tokenizer if we already trained it
tokenizer = build_tokenizer(
    tokenization_config, token_config, data_config.training_file_local
)

for item in iter_stories(tokenization_config, data_config.training_file_local):
    sentences = re.split(r"[.\n!?]+", item)
    for sentence in sentences[:2]:
        word_count = len(sentence.split(" "))
        encodings = tokenizer.encode(sentence)
        print(f"{sentence} has {word_count} words and {len(encodings)} tokens:")
        print(f"  {tuple(zip(encodings.tokens[:3], encodings.ids[:3]))}")
    break
```

```
Found it in /Users/sanch/Obsidian/2025-
spring/Notes/242B/Homeworks/hw3/artifacts/shared/tokenizers/tinystories_bpe_meta
space_5000_1000000.json, loading from there
Once upon a time there was a little boy named Ben has 11 words and 11 tokens:
  ((' Once', 363), (' upon', 372), (' a', 191))
 Ben loved to explore the world around him has 9 words and 8 tokens:
  ((' Ben', 480), (' loved', 442), (' to', 198))
```

The above print / display statements illustrate the tokenization process on the first 3 sentences of the first story

1.2 (b) Create the dataset

We concatenated tokenized stories into one long stream, appending after every story, and then chunked that stream into non-overlapping windows of length 128. For each chunk, the input is tokens t_0, …, t_{L-1} and the target is the same chunk shifted by one position, which implements standard next token prediction.

The results were: - **Train:** Train has 37,781,992 tokens and 295,171 samples - **Validation:** Train has 5,184,076 tokens and 40,500 samples

[ ]:
```
train_token_count = count_tokens(
    tokenization_config, tokenizer, data_config.training_file_local
)
valid_token_count = count_tokens(
    tokenization_config, tokenizer, data_config.validation_file_local
)

train_token_memmap_path = build_token_memmap(
    tokenization_config,
    token_config,
    tokenizer,
    data_config.training_file_local,
    train_token_count,
)
valid_token_memmap_path = build_token_memmap(
    tokenization_config,
    token_config,
    tokenizer,
    data_config.validation_file_local,
    valid_token_count,
)

training_samples = int(train_token_count / global_training_config.
 ↪context_length)
display(f"Train has {train_token_count} tokens and {training_samples} samples")

validation_samples = int(valid_token_count / global_training_config.
 ↪context_length)
display(f"Validation has {valid_token_count} tokens and {validation_samples}␣
 ↪samples")

train_dataset = TokenChunkDataset(
    train_token_memmap_path, train_token_count, global_training_config.
 ↪context_length
)
valid_dataset = TokenChunkDataset(
    valid_token_memmap_path, valid_token_count, global_training_config.
 ↪context_length
)
```

```
'Token count for stories is 189162345'

'Output train memmap at /Users/sanch/Obsidian/2025-spring/Notes/242B/Homeworks/
 ↪hw3/artifacts/tokenizers/train_tokens_metaspace_1000000_v3000.bin'
```

[ ]:
```
PosixPath('/Users/sanch/Obsidian/2025-
spring/Notes/242B/Homeworks/hw3/artifacts/tokenizers/valid_tokens_metaspace_1000
000_v3000.bin')
```

1.3 (c) Model

We used a pre norm decoder only transformer with sinusoidal positional encoding and tied input/output embeddings. The data flow is:

Input token IDs → Token embedding → + Sinusoidal position encoding → Transformer blocks (causal self-attention + feed-forward) → Final layer norm → Tied output projection → Next token logits

Each transformer block uses multi-head causal self attention with `scaled_dot_product_attention(..., is_causal=True)`, followed by a GELU feed forward network, residual connections, and layer normalization. Weight tying means the output projection reuses the token embedding matrix, which is standard in language models and keeps the parameter count lower.

We trained a total of 17 different models. The final model's parameter count was **46,368,768**. All the model architectural details are summarized in the table below. and the 46M model whose results we will be discussing is called xlarge-plus

[118]:
```
all_trained_models = pd.DataFrame(
    {
        **dict(results[k].config),
        "parameter_count": results[k].parameter_count,
        "real_name": k,
    }
    for k in results.keys()
)
display(
    all_trained_models[
        [
            "real_name",
            "parameter_count",
            "max_steps",
            "warmup_steps",
            "d_model",
            "n_heads",
            "n_layers",
            "d_ff",
            "batch_size",
            "learning_rate",
        ]
    ].head(10)
)
```

```
                 real_name  parameter_count  max_steps  warmup_steps  \
0  1_base_steps=1k_vocab=3k          1512000       1000            80
1         2_base_steps=100k          1512000     100000            80
2              3_large-plus         11799552     100000          1000
3     4_large-plus_vocab=5k         12567552     100000          1000
4             5_xlarge-plus         46368768     100000          1000
```

```
5  6_large-plus_context=512         12567552     100000          1000

   d_model  n_heads  n_layers  d_ff  batch_size  learning_rate
0      160        5         4   480          16           0.00
1      160        5         4   480          16           0.00
2      384        6         6  1536          16           0.00
3      384        6         6  1536          16           0.00
4      768       12         6  3072          64           0.00
5      384        6         6  1536          16           0.00
```

1.4 (d) and (g) Training & Experimentation

The first run we did was only about 600 steps, and the model made absolutely no sense. So, we scaled it in the next attempt up to 1,000. The validation loss continued to decrease, so we scaled up to 100,000 steps, and the models demonstrated learning in terms of:

- Better coherence across the entire story
- Understanding segues & transitions, endings, and general story structure
- Correct grammar

However, the models still had frequent typos, and would demonstrate entity confusion: if person A did task B, then the next sentence assumed person B performed that task. It also struggled a lot with direct speech, consistently forgetting who said what.

So, we started to scale up - first the model's architecture, then the vocabulary size, and finally the context window. At each step, we also continued to expand the model's overall dimension. The model's output quality drastically improved, indicating that the quality is definitely limited by the training budget.

The final model took processed a total of 819,200,000 tokens over 43 minutes, averaging about 315,076 tokens processed per second. We used the A100 GPU available via Colab.

1.4.1 Other Model Information

- We implemented a cosine decay for the learning rate decay
- Throughout the training process, we ended up saving the step-level losses and perplexity values
- For validation, we ran validation comparisons every 100 or 1000 steps (depending on the modes size), and saved to JSON
- The initial loss we got across all models was around 8.02 - 8.03, which is about equal to ln(3000), where 3000 is our vocabulary count.
- Below code is the training and validation losses for 5 of the models we ran. Some fun jumpss you can see (especially in the **validation graph**):
    1. Orange -> Green -> Purple is the effect of a larger architecture
    2. Green -> Red is the effect of increasing the vocabulary size
    3. Red -> Brown is the effect of context size.

Based on the results, as well as reviewing some of the samples that were generated across all the models, we observed the following:

1.4.2 Model Size

- Loss and perplexity improve by quite a lot when we increase the total number of parameters of the model
- The model better understands different aspects of the story more, like transitions, beginnings and endings.

1.4.3 Vocab Size

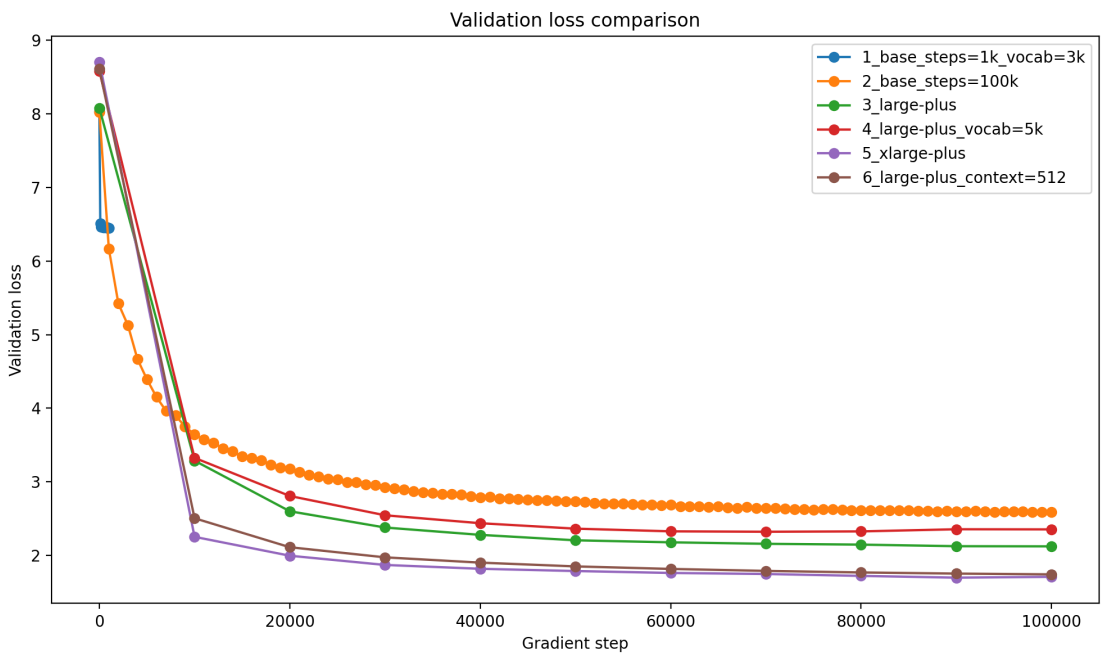
- Spelling mistake incidences reduced
- The training loss doesn't improve by a huge quantity
- Cross entropy losses are not comparable across vocabularies however because the size of the prediction space is changing. Cross-entropy measures how surprised our model is by the correct next token across all possible vocab tokens. So, if the vocab is changed, then there are a different number of choices available, which means that for two models with different vocabs, the losses are on a totally different scale.

1.4.4 Context Length

- This was a dramatic reduction in the training and validation loss

[100]:
```
plot_training_curves(
    {k: asdict(v) for k, v in results.items()}, run_config.plots /␣
 ↪"training_loss.png"
)
plot_validation_curves(
    {k: asdict(v) for k, v in results.items()}, run_config.plots /␣
 ↪"validation_loss.png"
)

display(Image(run_config.plots / "training_loss.png"))
display(Image(run_config.plots / "validation_loss.png"))
```



And here is the training loop for the final model

[121]:
```
configs = [
    (
```

```
        ModelConfig(
            name="xlarge-plus",
            d_model=768,
            n_heads=12,
            n_layers=6,
            d_ff=3072,
            batch_size=64,
            learning_rate=2.5e-4,
            weight_decay=0.1,
            warmup_steps=1000,
            max_steps=100000,
            use_amp=torch.cuda.is_available(),
        ),
    )
]
```

[ ]:
```
train_dataset = TokenChunkDataset(
    train_token_memmap_path, train_token_count, global_training_config.
 ↪context_length
)
valid_dataset = TokenChunkDataset(
    valid_token_memmap_path, valid_token_count, global_training_config.
 ↪context_length
)

# create the results object
results: dict[str, dict] = {}

# use GPU if available
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

for config in configs:
    metrics_path = run_config.metrics / f"{config.name}.json"
    if metrics_path.exists():
        results[config.name] = json.loads(metrics_path.
 ↪read_text(encoding="utf-8"))
        continue
    result = train_model(
        run_config,
        global_training_config,
        config,
        tokenizer,
        train_dataset,
        valid_dataset,
        device=device,
    )
    results[config.name] = result
```

```
    save_json(result, metrics_path)

    if device.type == "cuda":
        torch.cuda.empty_cache()
```

1.5 (e) Generative Sampling

[ ]:
```
generations = load_generations(
    "/Users/sanch/Obsidian/2025-spring/Notes/242B/Homeworks/hw3/artifacts/runs/
 ↪hw3-submission/metrics/samples/new-generations.json"
)
```

The table below illustrates different kinds of generated text based on the prompt, across three different model configurations: vocab=500k, context=128, and the large-plus, xlarge, and xlarge-plus architectures.

We tried this across 3 different temperature and top_k sampling strategies: - **(temp=0.7, top_k=30)**: these are the results we've displayed below - **(temp=0.5, top_k=10)**: this was our "conservative" generator - **(temp=0.8, top_k=50)**: this was our most "creative" generator

Long-span semantic cohesiveness is observed! Grammar is really good as well. Sometimes the model struggles with direct speech, but the quality is pretty coherent, and shows good overall structure, even though certain events that are described are logically impossible. We do observe a lot of children's story tropes as well - learning lessons, good morals and habits, etc.

[124]:
```
table = generations_to_dataframe(generations, temperature=0.7, top_k=30)
display(table.head())
```

```
model                                                                  ␣
 ↪                                                                     ␣
 ↪                                                                     ␣
 ↪                                                                     ␣
 ↪                                                                     ␣
 ↪                                                                     ␣
 ↪                                                                     ␣
 ↪                                                                     ␣
 ↪                                                                     ␣
 ↪                                                            large-plus  \
prompt
```

A cat named Mia wanted to go outside. ␣
↪ ␣
↪ ␣
↪ ␣
↪ A cat named Mia wanted to go outside. Mia was happy and␣
↪excited. She put on her coat and went out.\nAs Mia walked, she saw a big,␣
↪yellow ball. She picked it up and started to play. She was having a lot of fun.
↪ Suddenly, a big wind came. It blew and ran to the park. It was scared and␣
↪didn't know what to do.\nMia went back to her house. She was happy to be out␣
↪of the park. Mia and her mom played on theater all day. They had so much fun␣
↪together.
Early one morning Early one morning␣
↪she saw a big red ball. She felt a bit scared, so she picked it up and gave it␣
↪a big hug.\nUnfortunately, the ball began to bounce around in two. She wanted␣
↪to have it, but it was too late.\nShe tried to reach it but it was too high.␣
↪Then, a friendly dog came by. The dog was very kind and it licked the ball␣
↪with its paws. The ball kept bouncing and until it was all better. \nThe ball␣
↪was so thankful that it had been able to play with the ball. She was so happy␣
↪that she had made it to the other side of the room. \nThe moral of this story␣
↪is that it is important to always be careful when trying to catch things. For␣
↪a little help from then on, no matter how hard or you get to enjoy the fun and␣
↪have fun.
One day, a group of friends went to the park. ␣
↪ ␣
↪ ␣
↪ One day, a group of friends went to the park. The sun was shining␣
↪bright. All the friends liked to play in the grass. They were very happy.\nAt␣
↪the park, the sun was shining. The friend saw a big tree. The tree had a lot␣
↪of leaves. The friend wanted to climb the tree. But, no matter how, the tree␣
↪was not safe.\nThe friends had to wait. The tree did not get the leaves. It␣
↪was very sad. The other friends did not want to play on the tree. They went␣
↪home and did not have fun. The day ended of the day, the tree was not a tree,␣
↪but awake.
Tom and his best friend Brian were playing with their cars. ␣
↪ Tom and his best friend Brian were␣
↪playing with their cars. They had many cars of different colors and sizes.␣
↪They were having a lot of fun with the cars. They took turns and forth,␣
↪zooming the car and laughing. Then, Tom and Brian had to go fast and Billy␣
↪would try to catch the car with him. He did not see the car that was hit on␣
↪the floor.\nTom was sad and angry. He wanted to play with the car too. So, Tom␣
↪had an idea. He grabbed thermad and threw it at Tom and Brian. They made␣
↪noises and had fun. They pushed thermader, and it fell to the floor. They ran␣
↪after it, laughing and having fun. They learned that sharing is caring is more␣
↪important than having fun.

```
model                                                                                                  ␣
 ↪                                                                                                      ␣
 ↪                                                                                                      ␣
 ↪                                                                                                      ␣
 ↪                                                                                                      ␣
 ↪                                                                                                      ␣
 ↪                                                                                                      ␣
 ↪                                                                                                      ␣
 ↪                                                                                                      ␣
 ↪                                                                                                      ␣
 ↪                                                                                                      ␣
 ↪                                                                                                      ␣
 ↪                                                                                                      ␣
 ↪                                                                                                      ␣
 ↪                                                                                                ␣
 ↪xlarge  \
prompt
A cat named Mia wanted to go outside.                                                                  ␣
 ↪                                                                                                      ␣
 ↪                                                                                                      ␣
 ↪                                                                                                      ␣
 ↪                                                                                                      ␣
 ↪                                                                                                      ␣
 ↪                                                                                                      ␣
 ↪                            A cat named Mia wanted to go outside. But the sky␣
 ↪was gloomy, and the rain was falling down. Mia felt sad.\nMia's mom said,␣
 ↪"Mia, let's find a way for the rain to go away." They walked and walked, but␣
 ↪they could not find a way to go outside.\nOne day, a kind girl named Lily saw␣
 ↪them. She said, "I can help you find a way to go outside!" Mia and her mom␣
 ↪went outside. Mia was excited and said, "Yes, please help me!" And they all␣
 ↪went to find a way to go inside the gloomy sky.\nThey found a door and opened␣
 ↪it. Inside, they found a big room with lots of toys and games. Mia and her mom␣
 ↪played with the toys and had a lot of fun. The wet day turned into a fun day␣
 ↪for Mia and her family.
```

```
Early one morning                                                                                  ␣
 ↪                                                                                                 ␣
 ↪                                                                                                 ␣
 ↪                                                                                                 ␣
 ↪                                                                                                 ␣
 ↪                                                                                                 ␣
 ↪  Early one morning he would wake up early and go to the park. Every morning,␣
 ↪he would go to the park to play and have fun with his friends.\nOne night,␣
 ↪when he woke up, a big, scary monster appeared in the park. Joe was scared and␣
 ↪started to cry. His mommy and daddy came running and said, "Don't worry, Joey,␣
 ↪I will help you get back to the park." \nMommy and Daddy took Joe to the park␣
 ↪and Joe was still crying for a long time. He was so scared he couldn't even␣
 ↪notice. He felt sad and angry until he saw a little girl crying. She had a big␣
 ↪bag and was crying. Mommy and Daddy were so worried and they went to Jane to␣
 ↪comfort her. They took her back to the park and used her hands to open the bag␣
 ↪of toys. Joe was so happy to have his special toy back and was no longer␣
 ↪afraid of the park.
One day, a group of friends went to the park.                                                      ␣
 ↪                                                                                                 ␣
 ↪                                                                                                 ␣
 ↪                                                                                                 ␣
 ↪                                                                                                 ␣
 ↪                                                                                                 ␣
 ↪                                                                                                 ␣
 ↪                                                                                                 ␣
 ↪                                                                                                 ␣
 ↪                      One day, a group of friends went to the park. They saw a␣
 ↪big tree with a swing. The friends wanted to play on the swings, but they were␣
 ↪too high.\nA bird saw them and said, "I can help you get on the swings." The␣
 ↪friends were happy and said, "Yes, please!" The bird flew up and pushed the␣
 ↪swing down. The friends all clapped and cheered.\nThe bird was very happy and␣
 ↪said, "Thank you, friends!" The friends played on the swing for the rest of␣
 ↪the day, and they all had a great day. And from that day on, the group was a␣
 ↪happier place.
```

```
Tom and his best friend Brian were playing with their cars.  Tom and his best␣
↪friend Brian were playing with their cars. They liked to make them zoom around␣
↪the room, making loud noises.\n"Vroom, vroom!" Tom said, as he pretended to be␣
↪a driver on the street. "Vroom, vroom, vroom!"\n"Vroom, vroom, I'm a driver!"␣
↪Tom said, as he waved his hand.\nThey drove their cars around the room, making␣
↪loud noises. "Vroom, vroom, beep!" Tom shouted as he zoomed his cars.\nBut␣
↪then, Tom saw a bright light. He wanted to see what it was. He got off his car␣
↪and pulled out a bright light. He looked at the light and saw something shiny␣
↪and round. It was a necklace!\n"Wow, look at this!" Tom said, as he reached␣
↪for the necklace. "I want to keep it!"\nHe showed Lily his necklace. "It is␣
↪very pretty, but it is also very expensive. You should not touch it. You could␣
↪get hurt or lost. You could have hurt yourself or someone else or a gift from␣
↪a grown-up. That is very dangerous and dangerous."\nTom felt sorry and ashamed.
↪ He knew he had done something wrong. He hugged Lily and said, "I'm sorry,␣
↪Lily. I was wrong. I should not have touched the necklace. I should have␣
↪listened to you and stayed away from it."\nLily said, "I forgive you, Tom. I␣
↪am glad you are sorry. You are

model                                                                          ␣
↪                                                                              ␣
↪                                                                              ␣
↪                                                                              ␣
↪                                                                              ␣
↪                                                                              ␣
↪                                                                              ␣
↪                      xlarge-plus
prompt
A cat named Mia wanted to go outside.                                          ␣
↪                                                                              ␣
↪                                                A cat named Mia wanted to go␣
↪outside. She walked to the door and saw a big box. Mia was very curious, so␣
↪she went inside the box. Inside, she found a lot of toys.\nMia played with the␣
↪toys and had lots of fun. Then, Mia saw her mom and dad come to the door. They␣
↪were happy to see Mia playing. Mia's mom and dad were surprised too. They all␣
↪played together and had a great day.
Early one morning                                                              ␣
↪                                                                              ␣
↪                                                                              ␣
↪                                                                              ␣
↪                                                                              ␣
↪                                                                              ␣
↪                                                Early one morning and he was␣
↪ready to go to the park again soon.
```

```
One day, a group of friends went to the park.                One day, a group of␣
↪friends went to the park. They were eager to play together. They saw a big␣
↪tree and wanted to climb it. They were very excited.\nAs they climbed the␣
↪tree, they saw a little bird. The bird was sad. The group wanted to help the␣
↪bird. The group said, "Let us help the bird!" So, the friends used the bird to␣
↪climb down.\nThe bird was happy and said, "Thank you, friends!" Then, the bird␣
↪sang a pretty song. The friends felt good for helping. They played and had fun.
↪ They were happy they climbed the big tree.
Tom and his best friend Brian were playing with their cars.                    ␣
↪      Tom and his best friend Brian were playing with their cars. Brian was a␣
↪bit ignorant, so he asked Tom, "What do I have to do?"\nTom thought for a␣
↪moment and then said, "I have something for you. I can fold your shirt and␣
↪make it look like a new car!" Brian was excited and he started to fold his␣
↪shirt. \nTom was amazed at how great Brian's shirt looked and felt so proud of␣
↪his special shirt. From then on, he and Brian always made sure their shirts␣
↪were folded and had lots of fun playing together.
```

1.5.1 Question (f)

Here are some simple experiment on the learned embeddings of the tokens. We can try to understand the learned semantics of what each token would be, but performing simple arithmetic between tokens, and finding the closets neighbors of the resulting direction

In the three examples: - Lucy - girl + boy: gives us boys names, kind of as expected - snake - bad + good: gives us worms, caterpillars, alligators, which are varying degrees of less intense than snakes - sun - hot + cold: gives us snow, cold, rain, wind!

[53]:
```
word_pairs = [
    [
        " Lucy",
        " girl",
        " boy",
    ],
    [
        " snake",
        " bad",
        " good",
    ],
    [
        " sun",
        " hot",
        " cold",
    ],
]

for pair in word_pairs:
    print(f"{pair[0]} - {pair[1]} + {pair[2]} =")
    vec = (
```

```
        get_token_vector(tokenizer, model, pair[0])
        - get_token_vector(tokenizer, model, pair[1])
        + get_token_vector(tokenizer, model, pair[2])
    )

    for neighbor in find_closest_tokens(tokenizer, model, vec, top_k=5):
        print(f"     {neighbor}")
    print()
```

```
Lucy -  girl +  boy =
     (' Lucy', 0.8005343675613403)
     (' Timmy', 0.7502890825271606)
     (' Tim', 0.7471717596054077)
     (' boy', 0.6885653734207153)
     (' Tim.', 0.6815358996391296)

snake -  bad +  good =
     (' snake', 0.6920951008796692)
     (' snake.', 0.48493337631225586)
     (' good', 0.47660502791404724)
     (' turtle', 0.4573569595813751)
     (' neck', 0.4455905258655548)

sun -  hot +  cold =
     (' sun', 0.764910101890564)
     (' wind', 0.501511812210083)
     (' sunsh', 0.4477587640285492)
     (' snow', 0.4441651403903961)
     (' sky', 0.44201546907424927)
```